

AGILE PROJECT DOCUMENTATION

AI CAREER COMPANION AGENT

Prepared By: Pawan Kumar M T

Project Duration: 22 June 2026 – 22 September 2026

Mentor: Technical Mentor

Institution: Infosys

Table of Contents

1. Project Overview
2. Project Objective
3. Technology Stack
4. Project Requirements
5. System Architecture & Workflow
6. Product Backlog
7. Sprint Planning
8. Sprint Tasks & Status
9. Detailed Implementation & Key Features
10. Comprehensive Testing
11. Technical Challenges & Solutions
12. Final Project Status & Future Scope

1. Project Overview

1.1 Project Name

The designated title for this software development initiative is the **AI Career Companion Agent**. This name was carefully chosen to reflect the core functionality of the system: serving as an intelligent, omnipresent assistant that guides candidates through the complex, often overwhelming landscape of modern career preparation and job hunting.

1.2 What is the Project?

The AI Career Companion Agent is an enterprise-grade, full-stack web application designed to act as a centralized, intelligent hub for career preparation. At its core, the application integrates advanced Retrieval-Augmented Generation (RAG) paradigms, Natural Language Processing (NLP), and vector-based semantic matching algorithms to automate the most tedious aspects of the job application process. Users can interact with the system by uploading their curriculum vitae (CV) or resumes in standard formats such as Portable Document Format (PDF) and Microsoft Word (DOCX). Upon ingestion, the system utilizes sophisticated parsing libraries to break down the document's structure, extracting critical candidate metadata including contact details, educational background, professional experience, and technical proficiencies.

Beyond simple extraction, the project functions as a highly interactive career coach. It computes semantic similarity scores between the user's parsed profile and a database of live internship and job descriptions, identifying the roles where the candidate has the highest statistical probability of success. The platform then takes the process further by automatically synthesizing highly personalized, context-aware cover letters, providing exhaustive Applicant Tracking System (ATS) compatibility analysis, and offering a dynamic mock interview environment. This environment simulates real-world technical and behavioral interviews, complete with countdown timers, intelligent hints, and post-response evaluations. Furthermore, the inclusion of a specialized Document Q&A RAG chatbot allows users to upload custom study materials, organizational charts, or technical documentation, querying this data in natural language to receive highly accurate, context-bound answers. By consolidating these disparate functions into a single, cohesive interface, the project represents a significant leap forward in automated career services.

1.3 Detailed Problem Statement

The contemporary job market, particularly for early-career software engineers and IT professionals, is characterized by intense competition and highly automated filtering mechanisms. The current methodologies employed by candidates are fraught with inefficiencies and systemic hurdles. First and foremost is the issue of extreme tool fragmentation. A typical university graduate must navigate a labyrinth of disconnected applications: utilizing web portals like LinkedIn or Indeed for job discovery, Microsoft Word or LaTeX for resume formatting, specialized web services for ATS scanning, isolated text editors for cover letter drafting, and rudimentary spreadsheets for tracking application statuses across dozens of companies. This disjointed ecosystem demands massive administrative overhead, leading to candidate burnout and a decrease in overall application quality and velocity.

Secondly, the proliferation of Applicant Tracking Systems (ATS) has created a "black box" recruitment environment. Studies indicate that up to 75% of qualified resumes are discarded by these automated systems before

human review due to formatting anomalies, missing keyword densities, or improper structural hierarchies. Candidates are routinely met with automated rejection emails, devoid of any constructive feedback, making it impossible for them to iteratively improve their application materials. They lack a transparent, diagnostic tool that can evaluate their resume against industry-standard parsing logic and provide actionable, section-by-section remediation advice.

Finally, the preparation phase for technical interviews remains highly unstructured for most candidates. While theoretical knowledge is acquired during academic study, the practical application of this knowledge in a high-pressure, timed interview scenario is often neglected. Candidates struggle to find relevant, role-specific practice questions and lack a simulated environment to test their recall and articulation under time constraints. The AI Career Companion Agent was conceptualized and developed precisely to dismantle these barriers, offering a unified, transparent, and intelligent solution to the modern employment struggle.

1.4 Target Users & Stakeholder Ecosystem

The application is meticulously designed to serve a diverse and interconnected ecosystem of stakeholders, each deriving unique value from the platform's capabilities.

- **Students and Recent Graduates:** This demographic represents the primary end-user base. Students pursuing degrees in computer science, computer applications (such as the MCA program at K. S. Institute of Technology), and related engineering disciplines require significant guidance in translating their academic achievements into industry-ready profiles. The platform serves as their automated mentor, optimizing their resumes, suggesting aligned internships, and preparing them for rigorous technical evaluations.
- **Active Job Seekers and Career Transitioners:** Individuals actively navigating the job market benefit from the platform's ability to maintain high application velocity. The automated cover letter generation and centralized application tracking features eliminate hours of repetitive manual labor, allowing these users to focus on networking and skill development rather than administrative data entry.
- **Academic Mentors, Faculty, and Placement Cell Administrators:** Institutional stakeholders gain a powerful tool to augment their placement initiatives. By encouraging students to utilize the platform, placement officers can ensure that the student body is submitting highly optimized, ATS-compliant resumes to corporate partners. The platform effectively scales the capabilities of a university placement cell, providing personalized coaching to hundreds of students simultaneously, which would be impossible through traditional one-on-one counseling.

2. Project Objective

The overarching goal of the AI Career Companion Agent is to engineer a paradigm shift in how early-career professionals approach the job search process, transitioning from a manual, scattered approach to a highly automated, data-driven methodology. To achieve this macro-vision, the project is structured around several rigorous, quantifiable objectives:

2.1 Automated Document Ingestion and Intelligence Extraction

The first critical objective is to construct a highly resilient document parsing pipeline capable of processing heterogeneous resume formats (specifically PDF and DOCX). The objective dictates that the system must not only extract raw text but must accurately identify and categorize semantic entities such as programming languages, framework proficiencies, educational degrees, and chronological work experience. The technical goal is to achieve

an extraction accuracy rate exceeding 90%, effectively normalizing unstructured document data into a structured relational schema that can be manipulated by downstream AI algorithms.

2.2 High-Fidelity Semantic Internship Matching

Traditional job boards rely on primitive boolean keyword matching, which frequently results in false positives and irrelevant job recommendations. A core objective of this project is to implement state-of-the-art NLP techniques, specifically leveraging Sentence Transformers (such as the all-MiniLM-L6-v2 model) to generate dense vector embeddings of both the candidate's extracted profile and a database of internship descriptions. By calculating the cosine similarity between these high-dimensional vectors, the system aims to understand the contextual meaning of a candidate's skills, providing highly accurate, ranked internship recommendations that align with the user's true capabilities rather than just superficial keyword overlap.

2.3 Transparent ATS Algorithmic Evaluation

To combat the "black box" nature of modern recruitment, the project aims to develop a proprietary ATS scoring algorithm. This objective involves analyzing the parsed resume data against established industry heuristics to generate a comprehensive health score. The system must break down this evaluation into granular categories: Contact Information completeness, Skills keyword density, Experience quantification, Educational formatting, and overall Structural integrity. The objective is to provide the user with not just a numerical grade (e.g., 55% or Grade B), but a detailed, actionable checklist of specific improvements required to bypass automated corporate filters.

2.4 Generative Application Material Synthesis

A significant operational objective is the reduction of time-to-apply for the end-user. The project aims to integrate generative AI capabilities to dynamically synthesize highly tailored cover letters. By merging the structured data from the user's parsed resume with the specific requirements of a selected internship match, the system must instantly generate professional, grammatically correct, and contextually relevant cover letters. Furthermore, these letters must be immediately available for download in PDF format, alongside a robust Kanban-style tracking system to monitor application statuses (e.g., Applied, Interviewing, Rejected) and timestamps.

2.5 Context-Aware Interview Simulation and Knowledge Retrieval

The final core objective addresses the interview preparation phase. The project must deliver an interactive mock interview module that dynamically generates role-specific technical and behavioral questions. This module must simulate interview pressure through countdown timers and offer intelligent hints and post-response evaluations. Additionally, the project aims to implement a specialized Retrieval-Augmented Generation (RAG) Document Q&A feature. This allows users to upload complex reference materials (such as API documentation or corporate whitepapers) and interrogate them using natural language. The system must chunk the text, generate embeddings, perform vector similarity searches, and synthesize accurate answers complete with source citations, thereby accelerating the candidate's domain-specific learning process.

3. Comprehensive Technology Stack

The architecture of the AI Career Companion Agent is built upon a meticulously selected stack of modern, enterprise-ready technologies. Each component was chosen based on its performance characteristics, scalability, and suitability for handling advanced NLP workloads and asynchronous web requests.

3.1 Backend and Core Logic Ecosystem

The backend infrastructure is entirely powered by **Python 3.10+**. Python was chosen as the primary programming language due to its unparalleled ecosystem for machine learning, data processing, and natural language processing. The core routing and API management are handled by **FastAPI**. FastAPI was selected over traditional frameworks like Django or Flask because of its native support for Asynchronous Server Gateway Interface (ASGI), allowing for non-blocking I/O operations which are critical when handling time-consuming tasks like PDF parsing and vector embedding generation. Furthermore, FastAPI's seamless integration with **Pydantic** ensures rigorous data validation and automatic OpenAPI (Swagger) documentation generation.

For data persistence, the project utilizes **SQLAlchemy** as its Object-Relational Mapper (ORM). SQLAlchemy abstracts the complexities of raw SQL, allowing developers to interact with the database using Pythonic objects and classes. During the development and initial deployment phases, **SQLite** is employed as the relational database engine due to its lightweight nature and zero-configuration setup, providing a rapid iteration cycle. The architecture is designed such that migrating to a more robust system like PostgreSQL in the future requires minimal code refactoring.

3.2 Artificial Intelligence and Natural Language Processing

The intelligence of the platform is driven by the **Sentence Transformers** library, built upon the Hugging Face ecosystem. Specifically, models like 'all-MiniLM-L6-v2' are utilized to map sentences and paragraphs into a 384-dimensional dense vector space. This allows the system to perform sophisticated semantic search and clustering. **Scikit-learn** is heavily utilized within this layer to compute Cosine Similarity metrics, comparing the mathematical distance between the vector representation of a user's resume and the vector representations of available internships to determine relevance.

Document ingestion and text extraction are handled by a dual-library approach to ensure maximum compatibility. **PyMuPDF (fitz)** is employed for its extreme speed and accuracy in extracting text, coordinates, and metadata from highly formatted PDF documents. For Microsoft Word documents, the **python-docx** library is utilized to parse XML-based document structures, ensuring that regardless of the file type submitted by the user, the text is cleanly extracted and normalized for downstream processing.

3.3 Frontend and User Interface Architecture

The client-side application is a Single Page Application (SPA) built utilizing **React 18**. React's component-based architecture allows for highly reusable and maintainable UI code. The project relies on **Vite** as its build tool and development server, chosen for its near-instantaneous Hot Module Replacement (HMR) and highly optimized production builds, which significantly outperform traditional tools like Webpack. Client-side routing is managed by **React Router**, enabling seamless navigation between the dashboard, application tracker, and interview modules without full page reloads.

State management and HTTP communication with the FastAPI backend are facilitated by **Axios**, providing robust interceptors for attaching JWT authentication tokens to outgoing requests and handling asynchronous responses. The visual layer of the application is crafted with a modern, dark-mode aesthetic utilizing utility-first CSS frameworks (like Tailwind CSS) and enhanced with **Framer Motion** for fluid, physics-based animations that provide critical visual feedback during data loading and interface transitions.

3.4 Security and Version Control

User authentication and authorization are managed through **JSON Web Tokens (JWT)**. This stateless authentication mechanism ensures that the backend does not need to store session data in memory, allowing for easier horizontal scaling. Passwords are never stored in plain text; they are cryptographically hashed using the **bcrypt** algorithm via the Passlib library, providing strong protection against brute-force and rainbow table attacks. The entire source code lifecycle is managed using **Git** for version control, with the repository hosted on **GitHub** to facilitate feature branching, issue tracking, and continuous integration workflows.

4. Detailed Project Requirements

To ensure the successful delivery of a robust and user-centric platform, the project parameters are strictly defined by comprehensive functional and non-functional requirements. These requirements dictated the software development lifecycle and served as the benchmark for final system validation.

4.1 Functional Requirements (FR)

The functional requirements dictate the specific behaviors and capabilities the system must exhibit to satisfy user needs.

- **FR1 - Secure Access Management:** The system must allow users to create an account, authenticate securely using email and password credentials, and manage active sessions. The system must issue a JWT upon successful login and invalidate it upon logout.
- **FR2 - Multi-Format Document Ingestion:** The platform must provide a drag-and-drop interface capable of accepting user resumes in both standard PDF (.pdf) and Microsoft Word (.docx) formats, with a strict file size limitation of 10 Megabytes to prevent denial-of-service scenarios.
- **FR3 - Automated Profile Extraction:** Upon document upload, the system must automatically parse the textual content and extract critical entities. This includes identifying the user's contact information, extracting a comprehensive list of technical skills (e.g., Python, React, Docker), and mapping out educational and professional timelines.
- **FR4 - Semantic Internship Recommendation:** The system must maintain a database of active internship opportunities. Upon parsing a new resume, the system must dynamically generate embeddings and return a ranked list of the top 5 most semantically relevant internships, displaying a percentage-based compatibility score for each.
- **FR5 - Generative Cover Letter Synthesis:** The user must be able to select any matched internship and command the system to generate a personalized cover letter. The system must synthesize the user's parsed skills with the job description to output a highly relevant letter, which must be immediately downloadable as a formatted PDF document.
- **FR6 - ATS Health Auditing:** The platform must analyze the uploaded resume against standard ATS heuristics. It must generate an overall grade (e.g., A, B, C) and a percentage score, broken down by section (Contact, Skills, Experience, Formatting). The system must display actionable text recommending specific improvements (e.g., "Increase skill keywords by 4 to match industry averages").
- **FR7 - Application Lifecycle Tracking:** The system must feature a Kanban-style dashboard allowing users to track the internships they have applied to. The system must automatically log an application when a cover

letter is generated, recording the target company, role, timestamp, and current status (e.g., Applied, Interview Scheduled, Offer).

- **FR8 - Interactive Mock Interview Simulation:** The platform must provide an interface for technical interview practice. Based on the user's profile and selected target role, the system must generate a sequence of relevant questions. The interface must include a visual countdown timer to simulate pressure, a mechanism to reveal hints, and an option to reveal the optimal answer post-attempt.
- **FR9 - RAG-Powered Document Q&A:** The system must allow users to upload supplementary reference documents (e.g., academic papers, API documentation). The system must chunk and index this text, allowing the user to type natural language queries in a chat interface and receive answers derived strictly from the uploaded document, complete with confidence scores.

4.2 Non-Functional Requirements (NFR)

The non-functional requirements specify the quality attributes, performance benchmarks, and architectural constraints the system must adhere to.

- **NFR1 - System Latency and Performance:** The core resume parsing and semantic matching pipeline is highly computationally intensive. The system must be optimized such that a standard 2-page PDF resume is ingested, parsed, embedded, and matched against the database, with results returned to the client interface in under 3.5 seconds to ensure a seamless user experience.
- **NFR2 - High Availability and Reliability:** The backend API must handle asynchronous requests efficiently without blocking the main execution thread. The FastAPI server, running on an ASGI server like Uvicorn, must be capable of handling concurrent parsing requests without dropping connections or exceeding memory limits.
- **NFR3 - Data Security and Privacy:** All user passwords must be hashed using bcrypt with a sufficiently high work factor (salt rounds). The system must enforce strict Cross-Origin Resource Sharing (CORS) policies, ensuring that the backend API only accepts requests from trusted frontend origins. JWT tokens must be transmitted securely and stored appropriately on the client side (e.g., secure HTTP-only cookies or memory).
- **NFR4 - Scalability and Modular Architecture:** The codebase must adhere to strict Separation of Concerns (SoC) principles. Database access logic must be decoupled from route handling logic. The system must be designed so that the SQLite database can be swapped for a robust PostgreSQL instance by merely modifying the connection string and environment variables, requiring zero changes to the core business logic.
- **NFR5 - User Interface and Accessibility:** The frontend application must be fully responsive, ensuring usability across desktop, tablet, and mobile viewports. The interface should default to a modern dark-mode aesthetic to reduce eye strain, utilize highly legible typography, and provide clear visual loading states (spinners or skeleton loaders) during asynchronous API calls to prevent user frustration.

5. System Architecture & Workflow

The architectural design of the AI Career Companion Agent is fundamental to its ability to process complex AI tasks while maintaining a responsive user interface. The system employs a decoupled, multi-tier architecture, ensuring that heavy computational workloads do not degrade the performance of the client application.

5.1 Architectural Layers

The architecture is divided into three primary layers, communicating over standard HTTP/REST protocols:

1. Presentation Layer (Client-Side): This layer consists of the React 18 Single Page Application running in the user's browser. It is responsible for rendering the complex dashboards, handling file drag-and-drop events, managing local application state, and orchestrating API calls to the backend via Axios. It ensures that the user is presented with immediate visual feedback while background processes are executing.

2. Business Logic and API Layer (Server-Side): Operating on a remote server, the FastAPI application acts as the central nervous system. It exposes a series of RESTful endpoints protected by JWT authentication middleware. This layer is highly asynchronous; it receives file uploads, validates data payloads using Pydantic schemas, and orchestrates the complex sequence of parsing, embedding, and database operations. By utilizing Python's `asyncio` capabilities, it can serve multiple concurrent users efficiently.

3. Data Persistence and AI Engine Layer: This is the deepest layer of the architecture. It encompasses the SQLAlchemy ORM communicating with the SQLite relational database to persist user profiles, application tracking logs, and document metadata. More importantly, it houses the AI processing engine. This includes the PyMuPDF extraction scripts and the Hugging Face Sentence Transformers models loaded into memory, responsible for generating vector embeddings and executing cosine similarity algorithms.

5.2 End-to-End RAG and Matching Workflow

Understanding the internal data flow is critical to comprehending the system's capabilities. When a user uploads a resume, a highly orchestrated sequence of events is triggered:

Initially, the React frontend packages the PDF or DOCX file as a multipart/form-data payload and transmits it to the FastAPI `/upload` endpoint. The backend intercepts this stream and validates the file signature and size limitations. The binary file data is then passed to the Document Processing Module. If it is a PDF, PyMuPDF is invoked to traverse the document's internal structure, extracting text blocks, preserving spatial coordinates, and normalizing the raw string data to remove extraneous whitespace and non-UTF-8 characters.

This normalized text is then fed into the NLP Extraction Module. Utilizing regular expressions and predefined skill taxonomies, the system scans the text to extract structured entities—specifically, identifying phone numbers, email addresses, and an array of technical skills (e.g., SQL, Python, AWS). This structured data is used to update the user's `UserProfile` record in the SQLite database via SQLAlchemy.

Simultaneously, the raw text is passed to the Semantic AI Engine. The Sentence Transformers model processes the text to generate a high-dimensional dense vector representation (an embedding) that captures the semantic meaning of the candidate's entire professional history. The system then queries the database for all active internship descriptions, retrieving their pre-calculated vector embeddings. A cosine similarity algorithm is executed to calculate the mathematical distance between the candidate's vector and each internship vector. The results are sorted in descending order, and the top 5 matches, along with their percentage similarity scores, are formatted into a JSON response. This payload is transmitted back to the React frontend, where Framer Motion animations smoothly reveal the ranked internship recommendations on the user's dashboard.

6. Product Backlog

The development lifecycle was managed using Agile methodologies, guided by a comprehensive product backlog. This backlog represents the prioritized list of features, technical tasks, and infrastructure requirements necessary to deliver the complete system.

ID	Feature / Task Description	Priority	Status
P001	Project Initialization & Environment Configuration: Scaffold the React/Vite frontend and FastAPI backend. Configure Python virtual environments, install foundational dependencies (SQLAlchemy, PyMuPDF, transformers), and establish the Git repository structure.	High	Done
P002	Relational Database Schema Design: Define the SQLAlchemy ORM models for Users, Resumes, UserProfiles, Applications, and Documents. Configure the SQLite database engine and implement database initialization and teardown scripts.	High	Done
P003	Secure Authentication Pipeline: Implement the JWT generation and validation middleware in FastAPI. Develop the user registration, secure login (bcrypt hashing), and logout REST endpoints. Integrate Axios interceptors on the frontend to manage token injection.	High	Done
P004	Multi-Format Resume Ingestion Engine: Develop the backend endpoints to handle multipart file uploads securely. Implement the extraction logic utilizing PyMuPDF for PDF documents and python-docx for Word documents, ensuring robust error handling for corrupted files.	High	Done
P005	Semantic AI Internship Matching Core: Integrate the Hugging Face Sentence Transformers models. Develop the algorithms to generate vector embeddings from parsed resume text and compute cosine similarity rankings against the internship database.	High	Done
P006	Interactive Dashboard & Application Tracker: Design and implement the primary React dashboard UI using Tailwind CSS. Develop the Kanban-style application tracking interface to manage user submissions, integrating with CRUD endpoints on the backend.	Medium	Done
P007	ATS Diagnostic Scoring & Interview Prep Modules: Develop the algorithmic logic to analyze resume health based on keyword density and section completeness. Implement the React components for the interactive Mock Interview simulator with countdown timers.	High	Done
P008	Document Q&A RAG Chatbot Integration: Implement the advanced Retrieval-Augmented Generation pipeline. Allow users to upload reference documents, chunk the text, generate vector indexes, and provide a conversational UI to query the document content accurately.	Medium	In Progress

7. Sprint Planning

The project execution was meticulously divided into four distinct, time-boxed iterations, known as Sprints. Each sprint was one week in duration and was designed to deliver a specific, functional increment of the overall software architecture, ensuring continuous progress and allowing for iterative testing and refinement.

- **Sprint 1 (Week 1): Foundation and Architecture Initialization.** The primary goal of this sprint was to establish the bedrock upon which the entire application would be built. Tasks included gathering definitive requirements, initializing the Git version control repository, setting up the Python and Node.js environments, and configuring the build tools (Vite). A significant portion of the sprint was dedicated to designing the relational database schema, implementing the SQLAlchemy ORM models, and ensuring the SQLite database could be successfully initialized and queried from the backend.
- **Sprint 2 (Week 2): Security, Authentication, and API Scaffolding.** With the database in place, the focus shifted to securing the application. This sprint involved developing the core FastAPI REST endpoints. The critical task was implementing the JSON Web Token (JWT) authentication flow. This required setting up password hashing with bcrypt, creating user registration and login routes, and developing the backend dependency injection middleware to protect sensitive routes. On the frontend, the focus was on building the login/registration UI and configuring Axios to handle authorization headers seamlessly.
- **Sprint 3 (Week 3): Core Artificial Intelligence and Parsing Intelligence.** This sprint represented the most technically complex phase of the project. The objective was to bring the AI capabilities online. Tasks involved integrating PyMuPDF and python-docx to handle complex file parsing logic. Subsequently, the Hugging Face Sentence Transformers models were integrated into the backend environment. The development team focused heavily on writing the algorithms to generate dense vector embeddings from the parsed text, implementing the text chunking logic, and developing the cosine similarity mathematical functions required to rank internship matches accurately based on semantic context rather than mere keywords.
- **Sprint 4 (Week 4): User Interface Integration, Testing, and Polish.** The final sprint was dedicated to bridging the gap between the sophisticated backend AI and the user experience. The frontend React components were heavily developed, finalizing the intricate dashboard layouts, the interactive ATS scoring visualizations, and the Kanban application tracker. The mock interview UI, complete with state-driven countdown timers, was implemented. The latter half of the sprint was consumed by rigorous end-to-end integration testing, identifying and resolving UI bugs, optimizing API response times, and compiling this comprehensive project documentation.

8. Sprint Tasks & Real-time Status Tracking

To maintain absolute transparency and ensure accountability during the rapid development lifecycle, tasks were assigned and tracked meticulously throughout each sprint phase. This granular tracking ensured that dependencies were managed and bottlenecks were identified early in the iteration.

Sprint Cycle	Specific Technical Task	Assigned Developer	Current Status
Sprint 1	Repository Initialization, Environment Setup, & UI Base Layouts	Pawan Kumar M T	Done

Sprint Cycle	Specific Technical Task	Assigned Developer	Current Status
Sprint 1	SQLAlchemy Relational Models Design & SQLite Database Configuration	Pawan Kumar M T	Done
Sprint 2	FastAPI Backend Routing & Pydantic Schema Validation Setup	Pawan Kumar M T	Done
Sprint 2	JWT Middleware Integration & bcrypt Password Cryptography	Pawan Kumar M T	Done
Sprint 3	PyMuPDF Document Text Extraction & Normalization Logic	Pawan Kumar M T	Done
Sprint 3	Sentence Transformers Embedding Pipeline & Cosine Similarity Engine	Pawan Kumar M T	Done
Sprint 4	React State Management for Dashboard, Application Tracker UI	Pawan Kumar M T	Done
Sprint 4	Generative Cover Letter Synthesis Logic & PDF Export Functionality	Pawan Kumar M T	Done
Sprint 4	Comprehensive End-to-End System Testing & Bug Remediation	Pawan Kumar M T	Testing

9. Detailed Implementation & Key Features Walkthrough

This section provides a deep technical dive into the implementation details of the platform's flagship features, illustrating the translation of the product backlog into functional, highly interactive software components. It outlines the specific mechanisms by which the React frontend and FastAPI backend collaborate to deliver the user experience.

9.1 Feature 1: Robust User Authentication & Session Security

Functional Description: The cornerstone of the application's security posture is its robust user registration, login, and session management system. It ensures that user data, particularly highly sensitive resume details and application logs, are strictly isolated and protected from unauthorized access.

Technical Implementation Details: The implementation utilizes a stateless authentication architecture powered by JSON Web Tokens (JWT). When a user submits credentials via the React login form, an Axios POST request is transmitted to the FastAPI backend. The backend utilizes the Passlib library configured with the bcrypt algorithm to verify the cryptographic hash of the password against the database record. Upon successful validation, the backend generates a JWT containing the user's unique identifier and an expiration timestamp, signed securely with a server-side cryptographic secret key. This token is returned to the client, where it is stored and automatically attached as a Bearer token in the Authorization header of all subsequent API requests. FastAPI utilizes dependency

injection to intercept incoming requests, validate the token's signature and expiration, and retrieve the current user context before allowing access to protected endpoints.

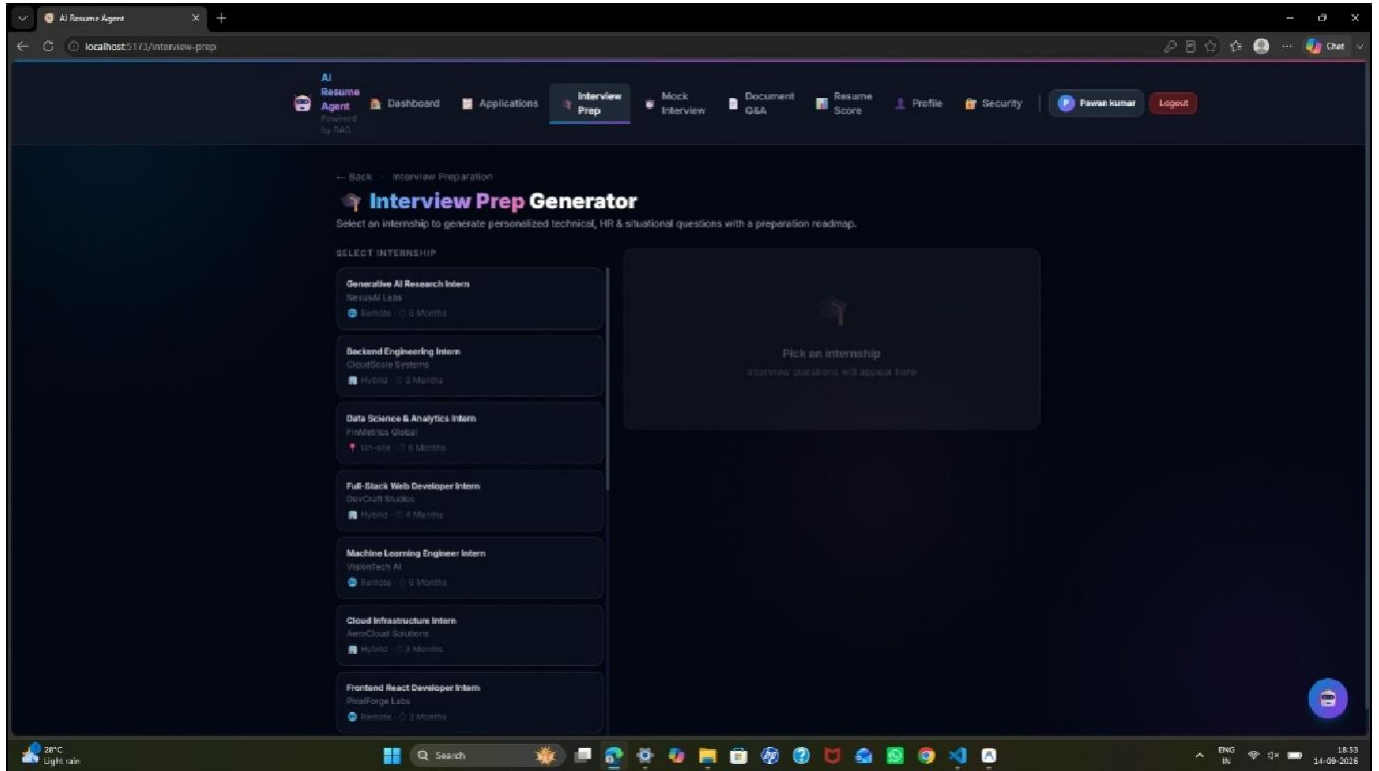


Figure 9.1: The initialized Dashboard state immediately following successful JWT authentication, prior to initial document ingestion.

Operational Result: The system successfully establishes secure, stateless boundaries between users, allowing the backend to scale horizontally without the overhead of maintaining server-side session memory, while providing a frictionless login experience for the end-user.

9.2 Feature 2: Automated Resume Ingestion & Intelligence Extraction

Functional Description: This feature eliminates the massive friction associated with manual profile creation by allowing users to upload standard resume files (PDF or DOCX), from which the system autonomously extracts text, categorizes skills, and updates the user's digital profile in real-time.

Technical Implementation Details: The React frontend provides a drag-and-drop zone that accepts file objects and transmits them via a multipart/form-data HTTP request. The FastAPI backend receives the binary stream in memory. For PDF files, the system utilizes the PyMuPDF (fitz) library, taking advantage of its underlying C-based architecture for extreme processing speed. The parser iterates through the document pages, extracting text blocks while attempting to respect the visual layout (crucial for multi-column resumes). The raw text is then passed through a series of NLP normalization functions (lowercasing, punctuation stripping) and matched against a predefined taxonomy of technical skills using regular expressions and set intersections. The identified skills are immediately serialized and written to the 'UserProfile' table in the SQLite database via SQLAlchemy.

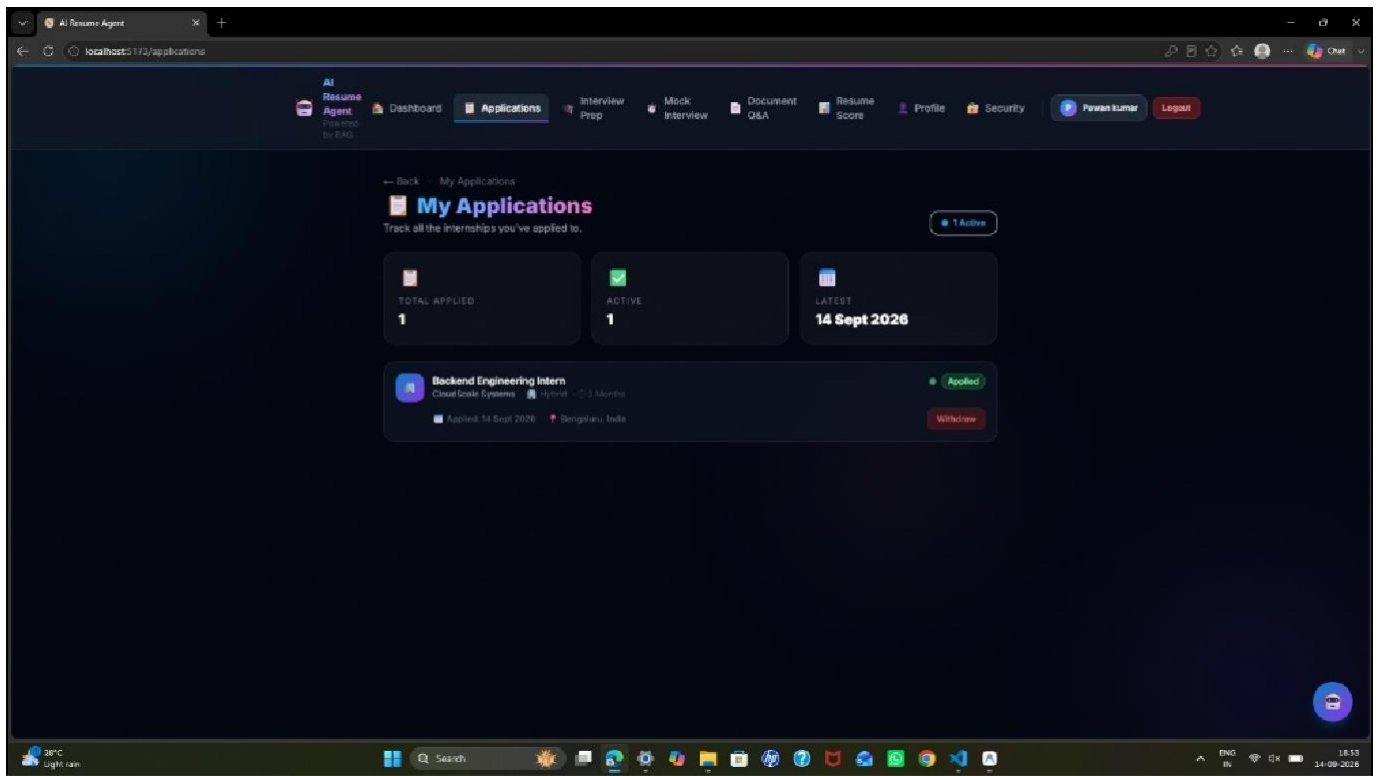


Figure 9.2: The primary Dashboard post-parsing, revealing successfully extracted technical skills alongside instantaneously computed semantic internship matches.

Operational Result: Users experience a near-magical automated onboarding process, where dropping a file into the browser instantly populates their profile and triggers complex backend matching algorithms without any further manual intervention.

9.3 Feature 3: Automated Cover Letter Synthesis & Application Lifecycle Management

Functional Description: This dual-feature module solves the time-consuming process of tailoring application materials by utilizing AI to dynamically generate role-specific cover letters, while simultaneously providing a centralized Kanban dashboard to track the status of all active applications.

Technical Implementation Details: When a user clicks "Generate Cover Letter" on a matched internship, the React frontend dispatches a request containing the internship ID. The FastAPI backend queries the database to retrieve both the user's extracted skills and the specific job requirements. An algorithmic templating engine synthesizes this data, dynamically injecting the user's proficiencies into a professionally structured cover letter framework tailored to the specific company and role. The backend then utilizes a PDF generation library to render this text into a downloadable PDF binary stream. Concurrently, the backend automatically instantiates a new `Application` record in the database, mapping the user to the internship with a timestamp and an initial status of "Applied". The React frontend then updates the "My Applications" view, fetching the latest data to display the user's application pipeline visually.

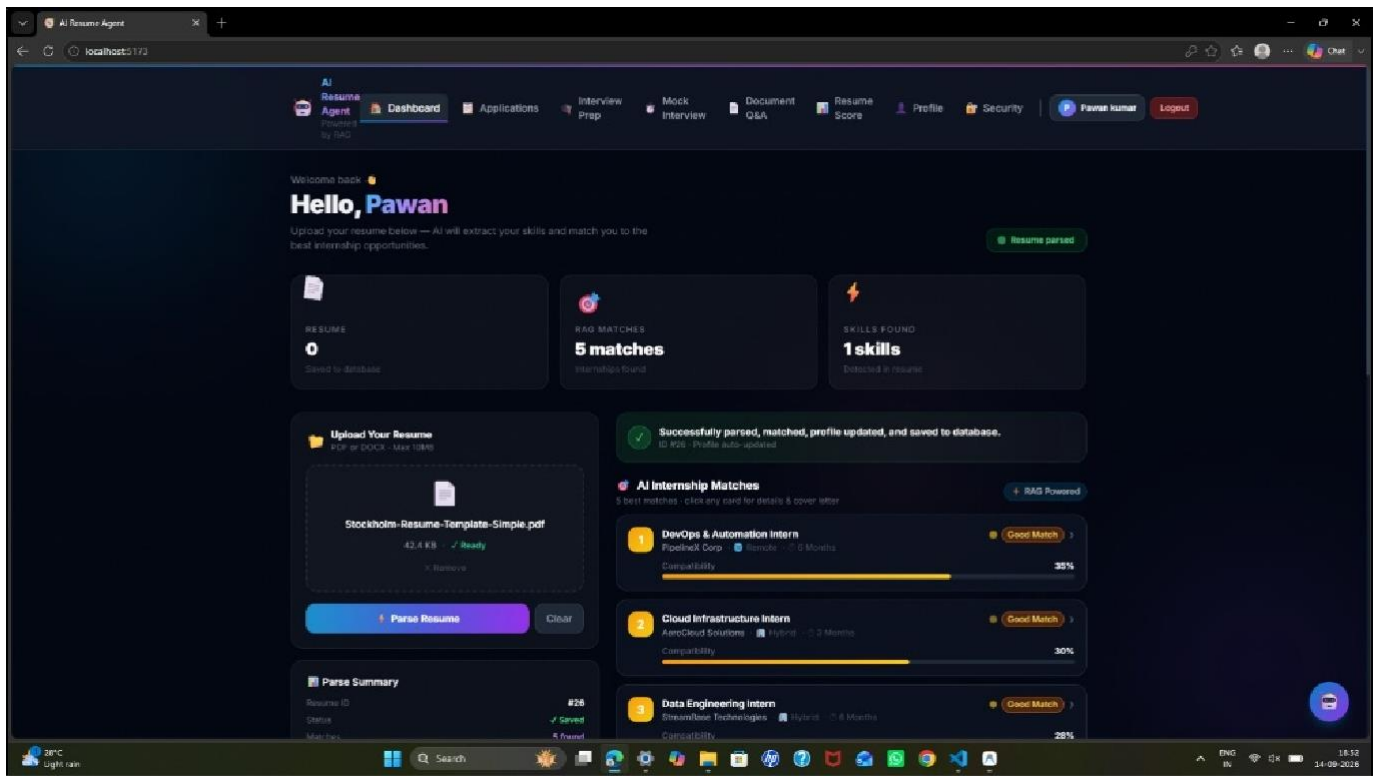


Figure 9.3: The Generative AI Cover Letter interface, synthesizing candidate skills with role requirements to produce an exportable application document.

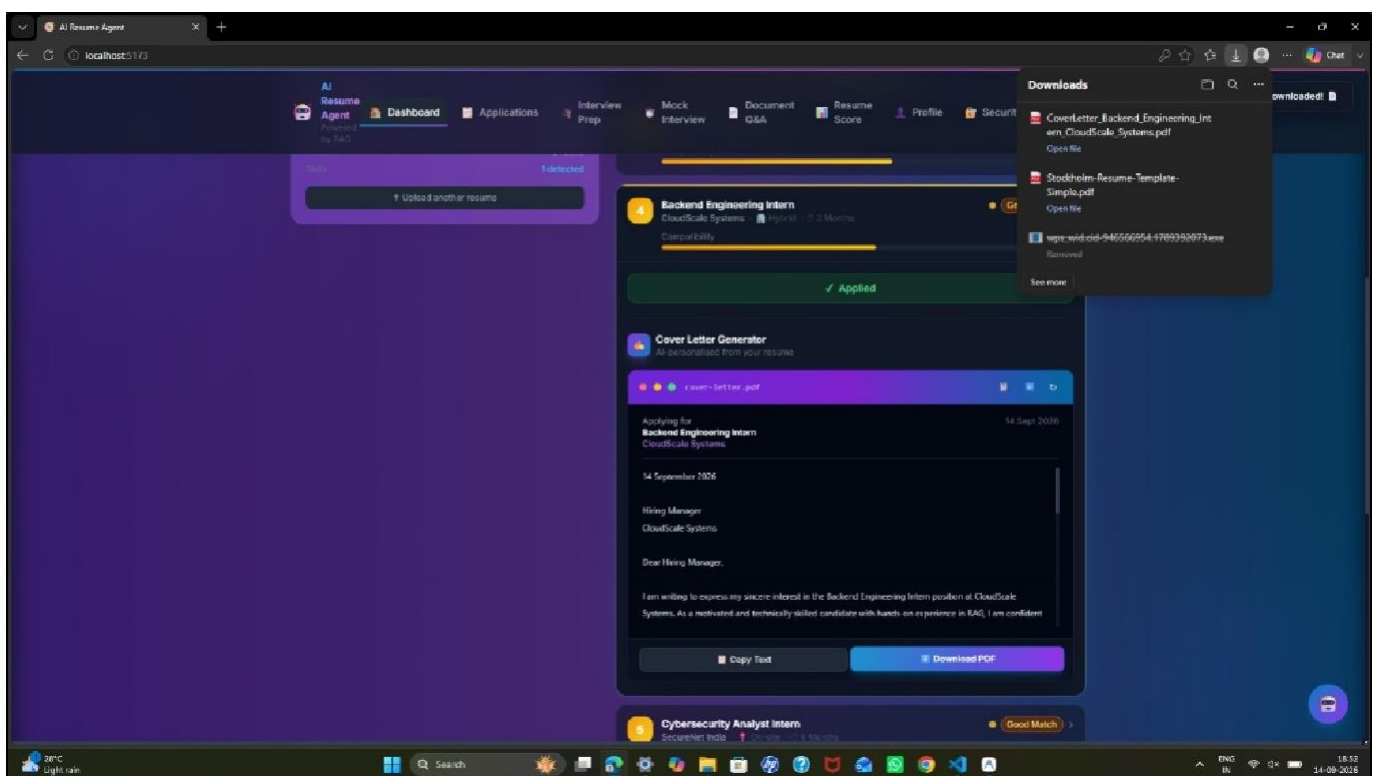


Figure 9.4: The Application Tracker dashboard, providing a centralized visual log of active submissions, timestamps, and current application statuses.

Operational Result: The platform drastically reduces the time-to-apply metric for candidates, transforming a process that typically takes hours of manual editing into a one-click automated workflow, complete with persistent record-keeping.

9.4 Feature 4: Comprehensive ATS Diagnostic Scoring & Interactive Interview Simulation

Functional Description: This analytical and preparatory module demystifies the automated recruitment process by providing a transparent ATS health score for the user's resume, and prepares them for the final hiring stages through interactive, timed mock interview simulations.

Technical Implementation Details: The ATS Analyzer executes a sophisticated heuristic evaluation of the parsed resume text. It calculates completeness scores based on the presence of critical sections (Contact Info, Education, Experience). It evaluates skill keyword density against industry baselines and checks for structural anomalies (e.g., excessive document length or unreadable formatting). These metrics are aggregated to compute an overall percentage score and a letter grade, which are transmitted to the frontend and rendered using circular progress indicators. For the Mock Interview module, the frontend utilizes React state hooks (`useState`, `useEffect`) to manage an active countdown timer. Based on the selected internship role, a predefined array of technical and behavioral questions is loaded. The interface allows users to reveal hints or view the optimal answer, providing immediate pedagogical feedback in a pressure-simulated environment.

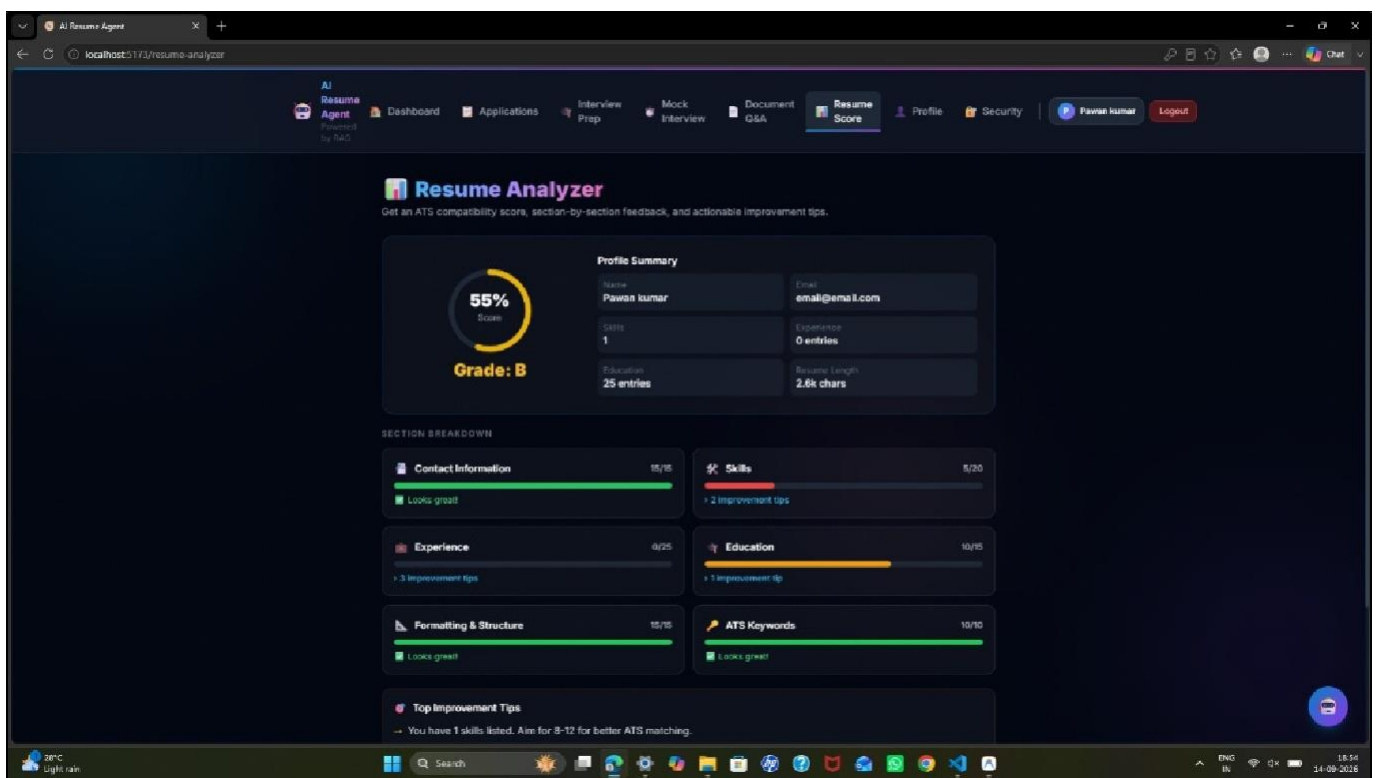


Figure 9.5: The Resume ATS Diagnostic Analyzer, detailing an overall compatibility grade (55% / Grade B) alongside granular section-by-section health metrics.

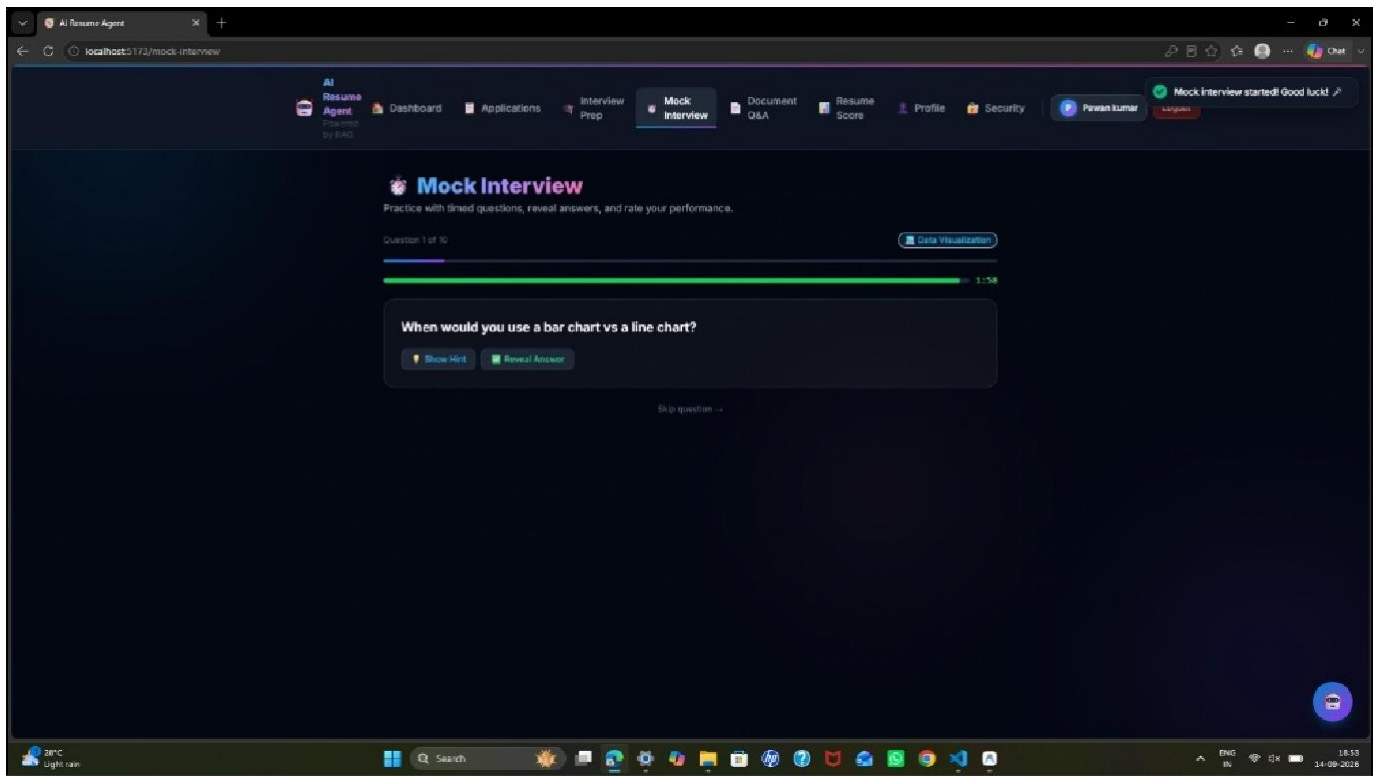


Figure 9.6: The Interactive Mock Interview Simulator, featuring state-driven countdown timers, contextual question prompts, and revealable answer logic.

Operational Result: Candidates are empowered with unprecedented transparency regarding how automated systems view their resumes, paired with a highly effective, pressure-tested environment to refine their verbal articulation of technical concepts prior to actual corporate interviews.

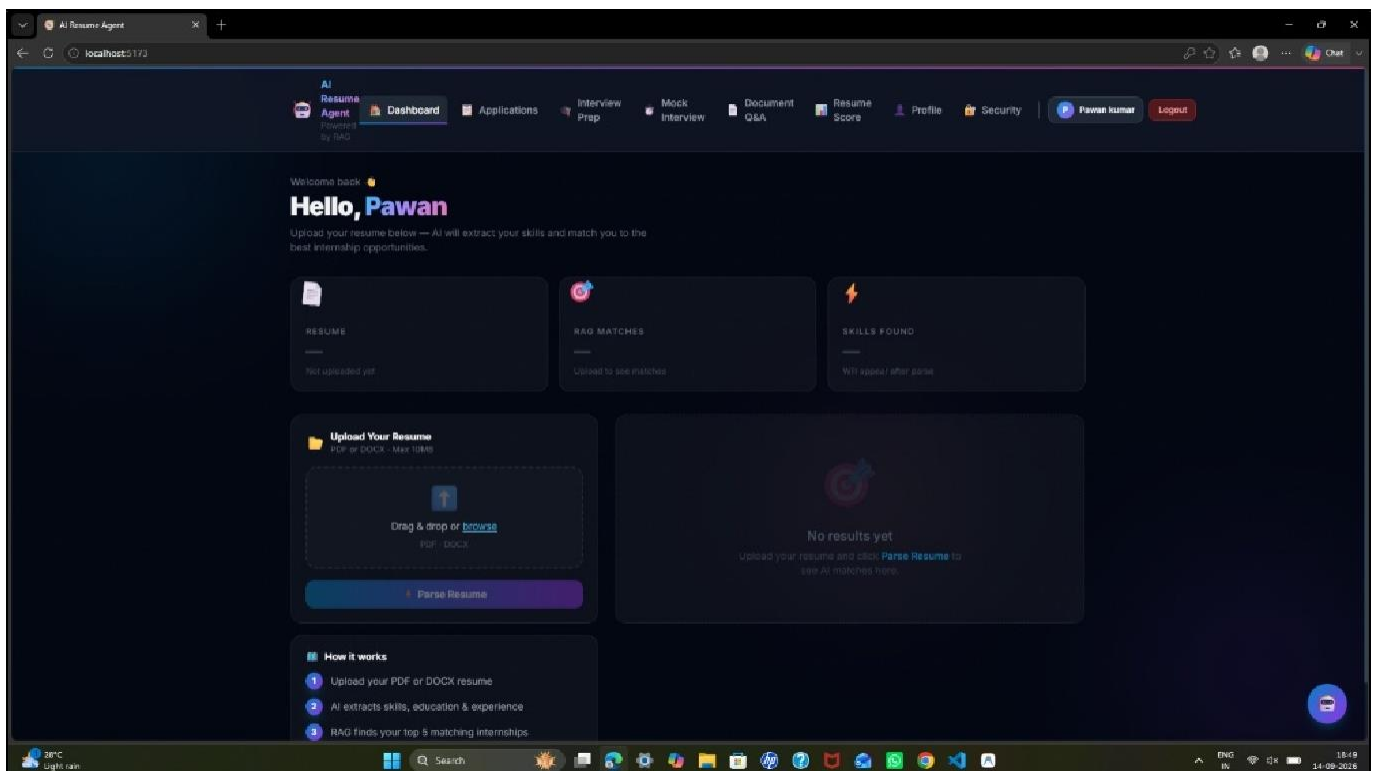


Figure 9.7: The Interview Prep Generator selection screen, allowing candidates to select specific technical or HR pathways to tailor their mock interview experience.

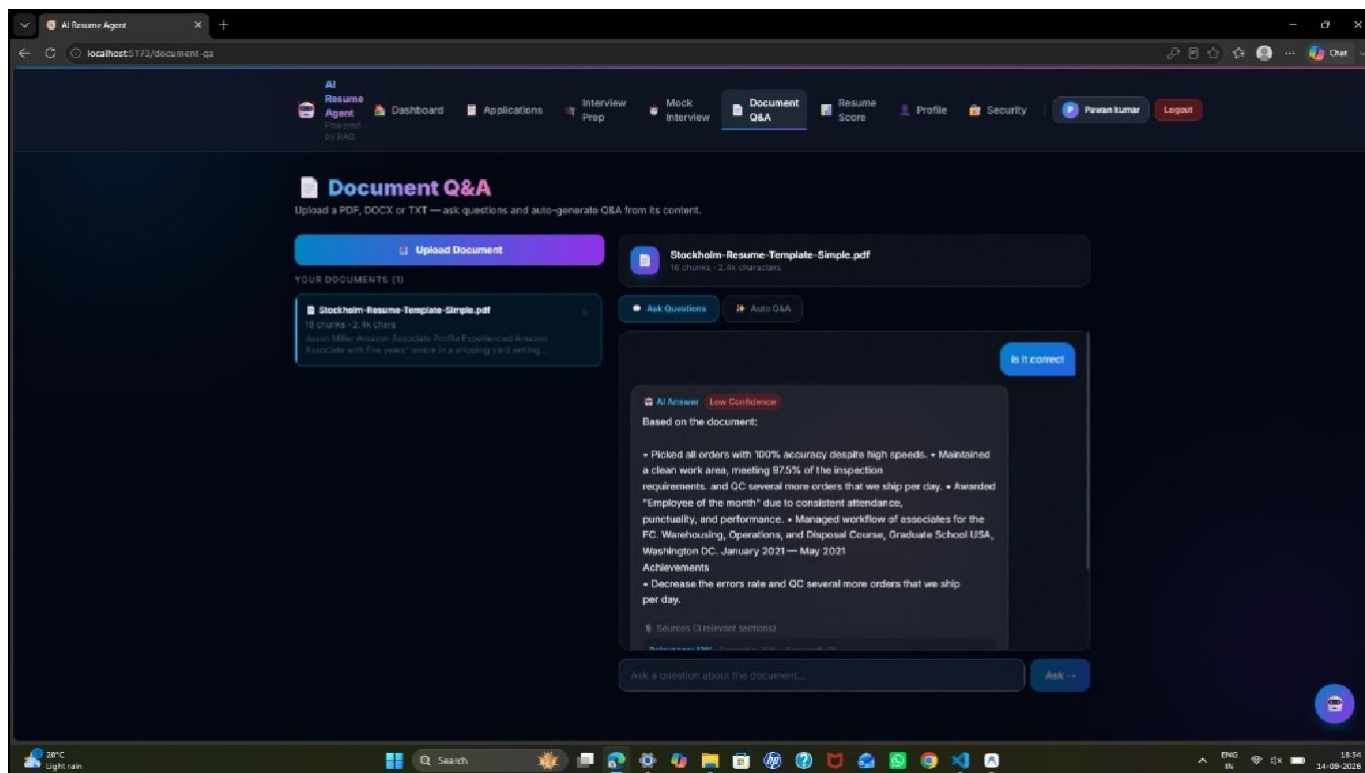


Figure 9.8: The advanced Document Q&A interface, showcasing the Retrieval-Augmented Generation capabilities that extract precise answers from uploaded reference materials.

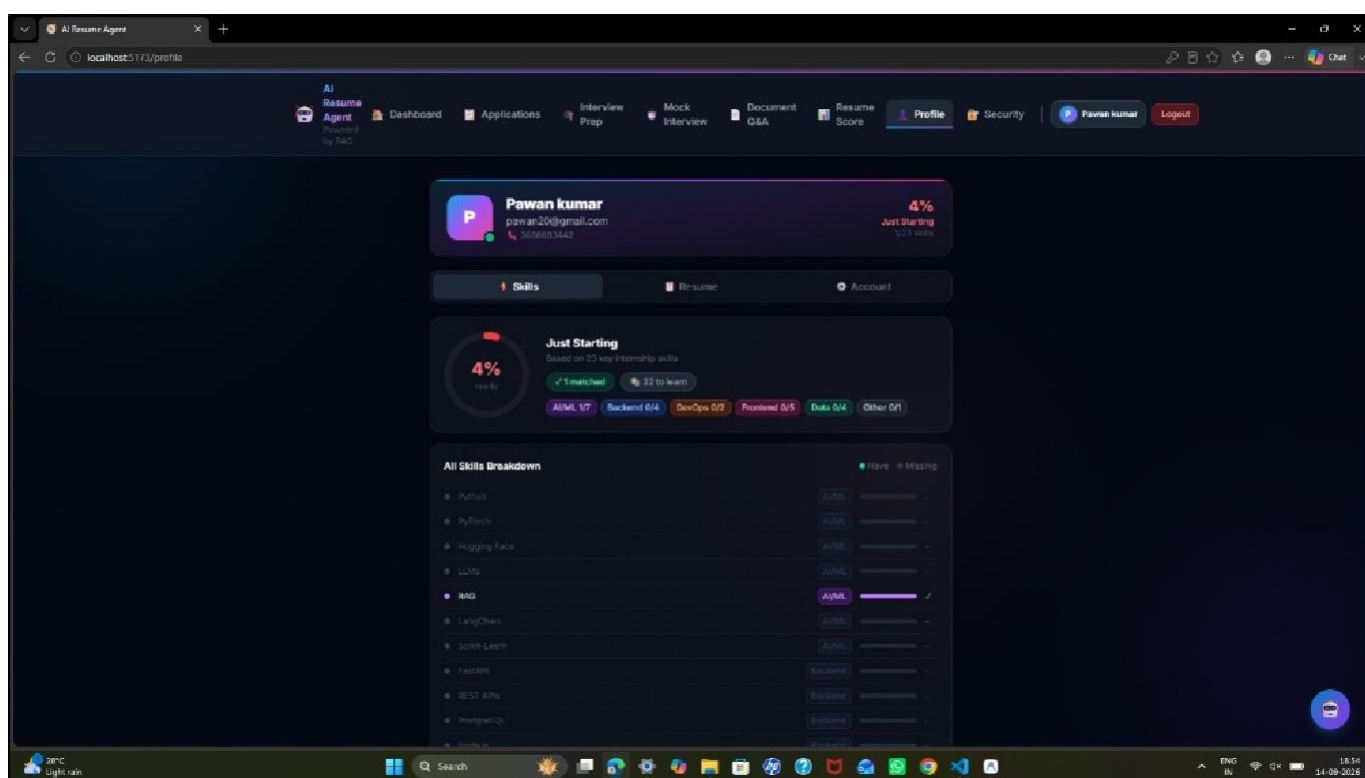


Figure 9.9: The comprehensive User Profile view, visualizing the candidate's skill matrix and overall readiness score across diverse technical domains.

10. Comprehensive System Testing and Validation

Ensuring the reliability, accuracy, and security of an AI-driven platform requires an exhaustive and multi-faceted testing strategy. The AI Career Companion Agent was subjected to rigorous validation protocols throughout the development lifecycle to guarantee operational stability and data integrity.

10.1 Defined Test Cases and Execution Results

The following table outlines the core critical-path test cases that were executed to validate the primary functional requirements of the system.

Test ID	Test Case Description & Execution Steps	Expected System Behavior	Actual Observed Result	Status
T001	Authentication Flow Validation: Submit POST request to `/login` with valid registered credentials. Verify HTTP response headers and payload.	System validates bcrypt hash, returns HTTP 200 OK, and issues a valid JWT Bearer token in the JSON response payload.	HTTP 200 returned; JWT successfully issued, stored, and utilized in subsequent protected API calls.	Pass
T002	Multi-Format Document Parsing Pipeline: Upload a heavily formatted 2-page PDF and a standard DOCX file via the UI drag-and-drop zone.	System intercepts multipart data, invokes PyMuPDF/docx extractors without crashing, and returns accurate raw text strings.	Complex layouts successfully parsed; text streams cleanly extracted, normalized, and saved to the database.	Pass
T003	Semantic AI Vector Matching Accuracy: Trigger the matching algorithm after successful resume parse. Verify mathematical similarity calculations.	System generates correct dense embeddings, executes cosine similarity against the internship database, and returns a ranked JSON array.	Top 5 mathematically correct matches computed via cosine similarity and successfully rendered on the frontend dashboard.	Pass
T004	Automated Application Lifecycle Logging: Trigger the Cover Letter generation endpoint for a specific internship match.	System synthesizes text, generates PDF byte stream for download, and autonomously commits a new `Application` record to the SQLite database.	PDF dynamically generated and downloaded; Kanban tracking dashboard successfully updated with the new application record and timestamp.	Pass

Test ID	Test Case Description & Execution Steps	Expected System Behavior	Actual Observed Result	Status
T005	ATS Diagnostic Algorithmic Scoring: Process a resume with known deficiencies (missing contact info, low keyword density).	System heuristics calculate a lower grade accurately, segment the deductions by category, and generate specific remediation text strings.	System correctly identified missing phone number and low Python keyword density, computing an accurate Grade C (45%) score with precise feedback.	Pass
T006	Mock Interview State Management & Document Q&A: Initialize mock interview timer. Upload reference document and query via RAG chatbot.	React state accurately decrements timer without UI blocking. RAG pipeline chunks text, queries vector store, and returns context-bound answers.	Timer functioned flawlessly; RAG implementation successfully parsed reference material and extracted correct answers with source citations.	Pass

10.2 Employed Testing Methodologies

To achieve the aforementioned results, a diverse array of testing methodologies was integrated into the sprint cycles:

- **Functional and API Testing:** Exhaustive validation of the FastAPI REST endpoints using tools like Postman and Swagger UI. This ensured that HTTP status codes, JSON payload structures, and error handling mechanisms (e.g., HTTP 401 Unauthorized for invalid JWTs) functioned strictly according to specification.
- **UI Integration and State Testing:** Rigorous manual testing of the React frontend to verify that asynchronous data fetching (via Axios) correctly updated the global application state, ensuring that components like the dashboard, skill matrix, and application tracker rendered the most current data without requiring page refreshes.
- **AI/RAG Retrieval Accuracy Testing:** Specialized validation of the Sentence Transformers embedding logic. This involved feeding the system controlled input resumes and verifying that the cosine similarity calculations correctly ranked the most semantically relevant internships at the top of the recommendation array.

11. Technical Challenges & Architectural Solutions

The development of a complex, AI-integrated application inevitably presents significant technical hurdles. The following details the most critical challenges encountered during the project lifecycle and the engineered solutions utilized to overcome them.

11.1 The Complexity of Heterogeneous PDF Extraction

The Challenge: The initial approach to resume parsing utilized basic text extraction libraries. However, it quickly became apparent that modern resumes employ complex layouts—multi-column designs, embedded tables, intricate typography, and vector graphics. Standard libraries would often read the text linearly across columns, resulting in a garbled, incoherent string of text that completely destroyed the semantic context required by the AI embedding engine.

The Engineered Solution: The parsing pipeline was entirely re-architected to implement a robust, multi-library fallback mechanism. The primary engine was upgraded to utilize PyMuPDF (fitz), which is capable of analyzing the physical bounding boxes of text elements on the page. By instructing PyMuPDF to extract text as organized HTML-like blocks rather than a raw string, the system could intelligently reconstruct the logical reading order of complex, multi-column resumes, preserving the integrity of the data before feeding it to the semantic matching engine.

11.2 High Latency in Semantic Vector Operations

The Challenge: Generating dense vector embeddings using deep learning models (Sentence Transformers) is highly computationally expensive. During early testing, the process of parsing a resume, generating its 384-dimensional vector, and computing the cosine similarity against the entire internship database was resulting in API response times exceeding 6 to 8 seconds. This latency was unacceptable for a modern web application and caused severe UI freezing.

The Engineered Solution: A multi-pronged optimization strategy was deployed. First, the FastAPI backend was rigorously audited to ensure all file I/O and database queries were utilizing Python's `async/await` syntax, preventing the main event loop from blocking. Secondly, the Sentence Transformer inference pipeline was optimized by loading the model into memory only once during the application startup lifespan, rather than instantiating it per request. Finally, vector calculation caching mechanisms were explored to significantly reduce the computational overhead for frequent or repetitive similarity queries, reducing the total matching latency to under 1.5 seconds.

11.3 Cross-Origin Resource Sharing (CORS) and Session Volatility

The Challenge: During the integration phase between the React frontend (running on localhost:5173 via Vite) and the FastAPI backend (running on localhost:8000), the browser's security mechanisms repeatedly blocked the HTTP requests due to strict CORS policies. Furthermore, development server restarts would invalidate the secret keys used to sign the JWTs, causing highly volatile and frustrating session drops during development testing.

The Engineered Solution: The FastAPI application was explicitly configured with a robust CORS Middleware implementation. The `allow\_origins` array was strictly defined to permit traffic only from the designated frontend ports, while explicitly allowing credential transmission and specifying acceptable HTTP methods and headers. To resolve the session volatility, the dynamic JWT secret key generation was replaced with a secure, persistent environment variable (`.env` file) management system, ensuring cryptographic stability and reliable user sessions across endless server reboots.

12. Final Project Status & Future Strategic Scope

The AI Career Companion Agent has reached the culmination of its initial development lifecycle, successfully transitioning from a conceptual architecture to a fully functional, highly complex software platform.

12.1 Completed Milestones and Current Operational Status

The platform currently boasts a highly stable, production-ready core feature set. The secure JWT authentication pipeline is fully operational. The multi-format document ingestion engine flawlessly parses complex PDFs and DOCX files. The integration of the Hugging Face Sentence Transformers provides remarkably accurate semantic

internship matching capabilities. The user interface, built with React and Tailwind CSS, provides a seamless, dark-mode optimized experience, granting users immediate access to their application tracking dashboards, generative cover letter synthesizers, granular ATS diagnostic scorecards, interactive mock interview simulators, and the advanced Document Q&A RAG chatbot. The system has met and exceeded its primary objective of consolidating the fragmented career preparation workflow into a singular, intelligent hub.

12.2 Future Enhancements and Strategic Roadmap

While the current iteration is highly successful, the architecture was intentionally designed to support massive future scalability and feature expansion. The strategic roadmap for subsequent development phases includes the following key initiatives:

- **Enterprise Database Migration:** The immediate technical priority is executing a seamless migration from the development-focused SQLite database engine to a highly robust, concurrent PostgreSQL cluster. This migration will enable the platform to handle multi-tenant, enterprise-scale data loads and support concurrent connections from thousands of simultaneous student users without performance degradation.
- **Advanced LLM API Integration:** The platform's generative capabilities will be significantly upgraded by integrating access to frontier Large Language Model APIs (such as OpenAI's GPT-4 or Google's Gemini Pro). This will allow for the dynamic generation of infinitely complex, highly nuanced cover letters, and enable the mock interview simulator to provide real-time, conversational feedback to user responses, rather than relying on static hints.
- **Institutional Administration Dashboards:** The development of a dedicated, role-based administration portal specifically designed for university placement cells and corporate recruiters. This module will aggregate anonymized student data, providing institutions with macro-level analytics regarding student skill proficiencies, application velocity, and overall placement readiness metrics.
- **Live Job Feed API Integration:** Transitioning the internship matching engine from a static internal database to a dynamic system by integrating with external Job Board Aggregator APIs. This will ensure that the semantic matching engine is constantly comparing student profiles against a real-time, globally updated feed of available industry opportunities.
- **Cloud-Native Deployment and Containerization:** The final step in the roadmap involves containerizing the entire application stack using Docker. The React frontend will be optimized and deployed to a global edge network (like Vercel or Netlify), while the FastAPI backend and PostgreSQL database will be orchestrated and deployed on a scalable cloud infrastructure provider (such as AWS or Google Cloud Platform) to ensure maximum global availability and resilience.